

Wireless performance of Zvw VS. RVV/CEVA

XinSheng Technology Co. LTD

Li gaoshan/Liang shengdun/Cai yuehong

Clock Cycles of 3 benchmark cases -- Zvw VS. RVV1.0

Clock cycles of Hardware Simulator			
NAME	Zvw Clock Cycle	Rvv Clock Cycle	RvvCycle/ZvwCycle rate
Conjugate Complex Multiply	11	30	2.7
Complex Dynamic Scaling MAC	31	224	7.2
Wideband Average Power	32	83	2.6

01

Measured cycles base on a RVV1.0 commercial VPU with "Zvw" extension

02

Both Zvw and RVV instruction are fully optimized according to VPU parallel architecture

Clock Cycles of wireless operator -- Zvw VS CEVA

Clock cycles of Hardware Simulator				
NAME	Zvw Clock Cycles	CEVA Clock Cycles	CEVA Cycle/Zvw Cycle rate	MAC equivalent cycle rate CEVA Cycle/Zvw Cycle
OSP (offset scalar product)	485	1695	3.5	1.75
NSP (normal scalar product)	399	1160	2.9	1.45

Zvw VPU: 22nm, 0.9 mm², 32GOPs (Int16) @ 1GHz, asm optimization

CEVA BX2: 22nm, 0.95 mm², 16GOPs (Int16) @ 1GHz, -Os optimization

Operators:

- OSP (Offset Scalar Product) is defined as the differential autocorrelation in the frequency - domain (total of 180 samples)
- NSP (Normal Scalar Product) is defined as the differential autocorrelation in the time - domain (total of 180 samples)

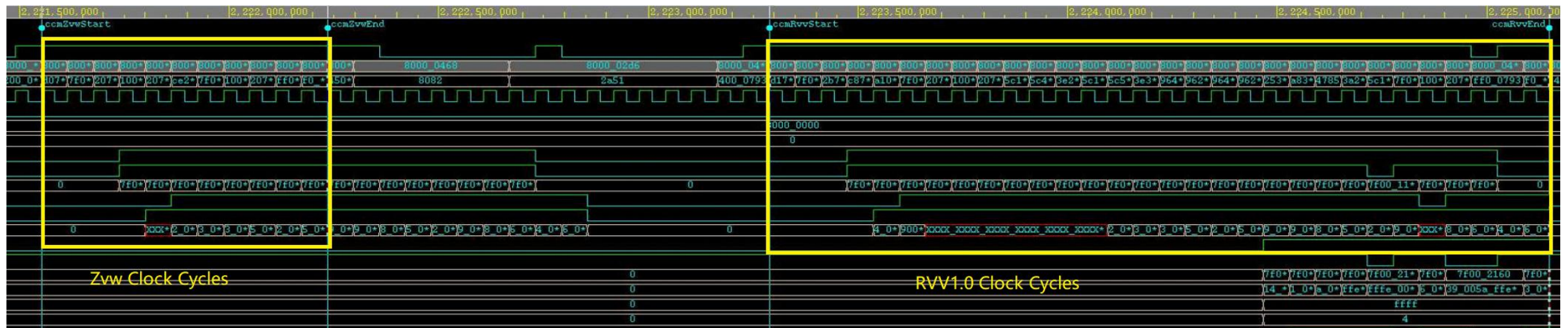
Conjugate Complex Multiply

Zvw version

```
av1 = 32;
vtypeE32 = TA | MA | M1 | E32;
asm volatile("vsetvli %[v1], %[av1], 208;":[v1] "=&r" (v1):[av1] "r" (av1));
asm volatile("vle32.v v2, (%[aAddr]);" ::[aAddr]"r"(pa));
asm volatile("vle32.v v1, (%[bAddr]);" ::[bAddr]"r"(pb));
asm volatile("vdscmulj.vv v4, v2, v1;");
asm volatile("vse32.v v4, (%[rZvmAddr]);" ::[rZvmAddr]"r"(pz));
asm volatile("vsync %[syncRd],%[rs2];":[syncRd]"=r"(syncRd):[rs2]"r"(rs2));
```

RVV1.0 version

```
av1 = 64;
vtypeE = TA | MA | M2 | E32;
asm volatile("vsetvli %[v1], %[av1], 209;":[v1] "=&r" (v1):[av1] "r" (av1));
asm volatile("vlm.v v0, (%[imageCcmMaskAddr]);":[imageCcmMaskAddr]"r"(pm));
vtypeE = TA | MA | M1 | E16;
asm volatile("vsetvli %[v1], %[av1], 200;":[v1] "=&r" (v1):[av1] "r" (av1));
asm volatile("vsub.vv v1, v1,v1;");
asm volatile("vle16.v v4, (%[aAddr]);":[aAddr]"r"(pa));
asm volatile("vle16.v v5, (%[bAddr]);":[bAddr]"r"(pb));
asm volatile("vmerge.vvm v2,v1,v4,v0;");
asm volatile("vmerge.vvm v4,v4,v1,v0;");
asm volatile("vsliddown.vi v2,v2,1;");
asm volatile("vmerge.vvm v3,v1,v5,v0;");
asm volatile("vmerge.vvm v1,v5,v1,v0;");
asm volatile("vsliddown.vi v3,v3,1;");
asm volatile("vmul.vv v5,v4,v1;");
asm volatile("vmul.vv v6,v2,v3;");
asm volatile("vmul.vv v7,v4,v3;");
asm volatile("vmul.vv v8,v2,v1;");
asm volatile("vadd.vv v1,v5,v6;");
asm volatile("vsub.vv v2,v8,v7;");
asm volatile("vslideup.vx v3,v2,%[num1];":[num1]"r"(num1));
asm volatile("vmerge.vvm v1,v1,v3,v0;");
asm volatile("vse16.v v1, (%[rRvvAddr]);":[rRvvAddr]"r"(pz));
asm volatile("vsync %[syncRd],%[rs2];":[syncRd]"=r"(syncRd):[rs2]"r"(rs2));
```



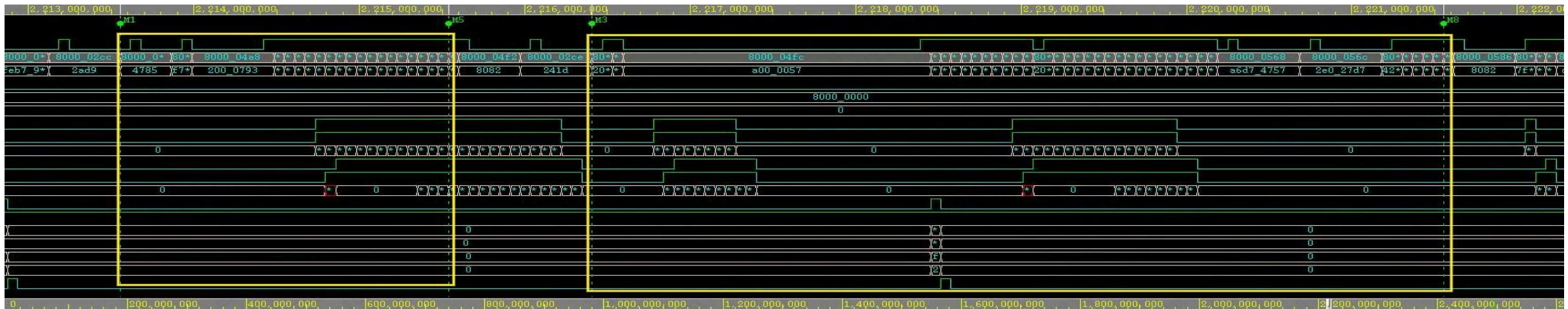
Wideband Average Power

Zvw version

```
av1 = 32;
vtypeE = TA | MA | M1 | E32;
asm volatile("vsetvli %[v1], %[av1],208;":[v1] "=&r" (v1):[av1] "r" (av1));
asm volatile("vle32.v v0, (%[a0Addr]);" ::[a0Addr]"r"(VM_SRC1_ADDR));//LSD
asm volatile("vle32.v v1, (%[gainShiftAddr]);" ::[gainShiftAddr]"r"(VM_GAIN_RVV_ADDR));
asm volatile("vsrl.vx v3, v2, %[num16];" ::[num16]"r"(16));//VALU2 IM(L)
asm volatile("vsll.vx v4, v2, %[num16];" ::[num16]"r"(16));//VALU2 RE(H)
asm volatile("vle32.v v9, (%[a1Addr]);" ::[a1Addr]"r"(VM_SRC1_ADDR + 32*4));//LSD
asm volatile("vsrl.vx v5, v4, %[num16];" ::[num16]"r"(16));//VALU2 RE(L)
asm volatile("vsrl.vx v13, v9, %[num16];" ::[num16]"r"(16));//VALU2 IM1(L)
asm volatile("vmul.vv v6, v5, v5;" ::);//VALU1 RE(L)*RE(L)
asm volatile("vsll.vx v14, v9, %[num16];" ::[num16]"r"(16));//VALU2 RE1(H)
asm volatile("vsra.vx v6, v6, %[mulShfit];" ::[mulShfit]"r"(0));//VALU2
asm volatile("vmul.vv v7, v3, v3;" ::);//VALU1 IM(L)*IM(L)
asm volatile("vsrl.vx v15, v14, %[num16];" ::[num16]"r"(16));//VALU2 RE1(L)
asm volatile("vsra.vx v7, v7, %[mulShfit];" ::[mulShfit]"r"(0));//VALU2
asm volatile("vmul.vv v17, v13, v13;" ::);//VALU1 IM1(L)*IM1(L)
asm volatile("vadd.vv v8, v7, v6;" ::[num16]"r"(16));//VALU1 RE(L)*RE(L)+IM(L)*IM(L)
asm volatile("vmul.vv v16, v15, v15;" ::);//VALU1 RE1(L)*RE1(L)
asm volatile("vsra.vv v11, v8, v1;" ::);//VALU2 vsra(RE(L)*RE(L)+IM(L)*IM(L))
asm volatile("vadd.vv v10, v17, v16;" ::);//VALU1 RE1(L)*RE1(L)+IM1(L)*IM1(L)
asm volatile("vsra.vv v12, v10, v1;" ::);//VALU2 vsra(RE1(L)*RE1(L)+IM1(L)*IM1(L))
asm volatile("vadd.vv v13, v11, v12;" ::);//VALU1
asm volatile("vsra.vx v14, v13, %[accShfit];" ::[accShfit]"r"(0));//VALU2
asm volatile("vredsum.vs v15, v14, v0;" ::);//VALU2
asm volatile("vmv.x.s %[result], v15;" ::[result]"=&r"(*pz));//VALU2
asm volatile("vsync %[syncRd],%[rs2];" ::[syncRd]"=&r"(syncRd):[rs2]"r"(rs2));
```

RVV1.0 version

```
asm volatile("vsetvli %[v1], %[av1],208;":[v1] "=&r" (v1):[av1] "r" (32));
asm volatile("vsub.vv v0, v0,v0;" ::);//VALU1
asm volatile("vle32.v v2, (%[a0Addr]);" ::[a0Addr]"r"(VM_SRC1_ADDR));//LSD
asm volatile("vsrl.vx v3, v2, %[num16];" ::[num16]"r"(16));//VALU2 IM(L)
asm volatile("vle32.v v1, (%[gainShiftAddr]);" ::[gainShiftAddr]"r"(VM_GAIN_RVV_ADDR));
asm volatile("vsll.vx v4, v2, %[num16];" ::[num16]"r"(16));//VALU2 RE(H)
asm volatile("vle32.v v9, (%[a1Addr]);" ::[a1Addr]"r"(VM_SRC1_ADDR + 32*4));//LSD
asm volatile("vsrl.vx v5, v4, %[num16];" ::[num16]"r"(16));//VALU2 RE(L)
asm volatile("vsrl.vx v13, v9, %[num16];" ::[num16]"r"(16));//VALU2 IM1(L)
asm volatile("vmul.vv v6, v5, v5;" ::);//VALU1 RE(L)*RE(L)
asm volatile("vsll.vx v14, v9, %[num16];" ::[num16]"r"(16));//VALU2 RE1(H)
asm volatile("vsra.vx v6, v6, %[mulShfit];" ::[mulShfit]"r"(0));//VALU2
asm volatile("vmul.vv v7, v3, v3;" ::);//VALU1 IM(L)*IM(L)
asm volatile("vsrl.vx v15, v14, %[num16];" ::[num16]"r"(16));//VALU2 RE1(L)
asm volatile("vsra.vx v7, v7, %[mulShfit];" ::[mulShfit]"r"(0));//VALU2
asm volatile("vmul.vv v17, v13, v13;" ::);//VALU1 IM1(L)*IM1(L)
asm volatile("vadd.vv v8, v7, v6;" ::[num16]"r"(16));//VALU1 RE(L)*RE(L)+IM(L)*IM(L)
asm volatile("vmul.vv v16, v15, v15;" ::);//VALU1 RE1(L)*RE1(L)
asm volatile("vsra.vv v11, v8, v1;" ::);//VALU2 vsra(RE(L)*RE(L)+IM(L)*IM(L))
asm volatile("vadd.vv v10, v17, v16;" ::);//VALU1 RE1(L)*RE1(L)+IM1(L)*IM1(L)
asm volatile("vsra.vv v12, v10, v1;" ::);//VALU2 vsra(RE1(L)*RE1(L)+IM1(L)*IM1(L))
asm volatile("vadd.vv v13, v11, v12;" ::);//VALU1
asm volatile("vsra.vx v14, v13, %[accShfit];" ::[accShfit]"r"(0));//VALU2
asm volatile("vredsum.vs v15, v14, v0;" ::);//VALU2
asm volatile("vmv.x.s %[result], v15;" ::[result]"=&r"(*pz));//VALU2
asm volatile("vsync %[syncRd],%[rs2];" ::[syncRd]"=&r"(syncRd):[rs2]"r"(0xFF));
```



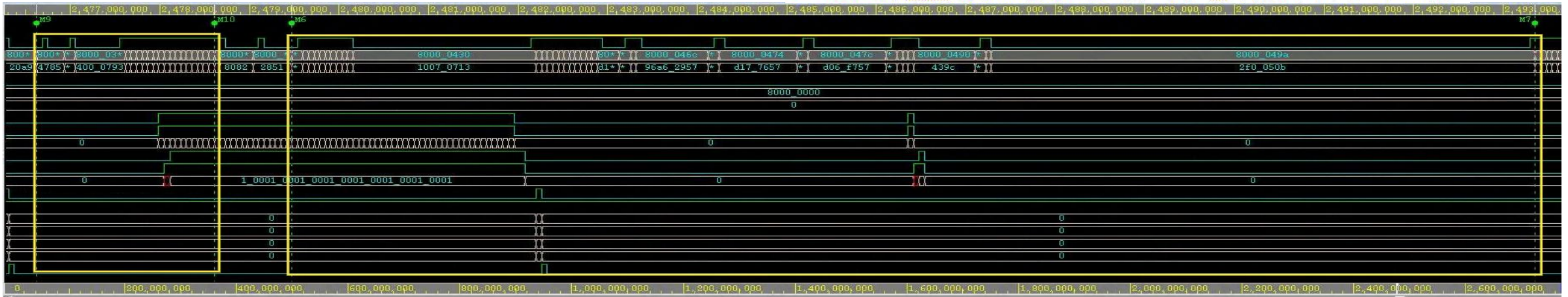
Complex Dynamic Scaling MAC

Zwv version

```
av1 = 64;
vtypeE = TA | MA | M2 | E32;
asm volatile("vsetvli %[v1], %[av1],209;:[v1] \"=&r\" (v1):[av1] \"r\" (av1));
asm volatile("vdsmacini.s %[gainShift];" ::[gainShift]"r"(gainShift));
asm volatile("vle32.v v0, (%[aCdsMAddr]);" ::[aCdsMAddr]"r"(pa));
asm volatile("vle32.v v2, (%[bCdsMAddr]);" ::[bCdsMAddr]"r"(pb));
asm volatile("vdscmacjo.vv v10,v0, v2;::");
av1 = 32;
vtypeE = TA | MA | M1 | E32;
asm volatile("vsetvli %[v1], %[av1],208;:[v1] \"=&r\" (v1):[av1] \"r\" (av1));
asm volatile("vdscrdsum.v v12,v10;::");
asm volatile("vmv.x.s %[result], v12;" :[result]"=&r"(*pz));
asm volatile("vsync %[syncRd],[rs2];:[syncRd]\"=&r\"(syncRd):[rs2]\"r\"(rs2));
```

RVV1.0 version

```
asm volatile("vsetvli %[v1], %[av1],208;:[v1] \"=&r\" (v1):[av1] \"r\" (1));
asm volatile("vmv.v.i v26, 0;");//VALU2
asm volatile("vsetvli %[v1], %[av1],209;:[v1] \"=&r\" (v1):[av1] \"r\" (64));
asm volatile("vle32.v v2, (%[aCdsMAddr]);" ::[aCdsMAddr]"r"(VM_SRC1_ADDR));//LSD
asm volatile("vsrl.vx v6, v2, %[num16];" ::[num16]"r"(16));//VALU2 IM0(L)
asm volatile("vle32.v v4, (%[bCdsMAddr]);" ::[bCdsMAddr]"r"(VM_SRC2_ADDR));//LSD
asm volatile("vsrl.vx v12, v4, %[num16];" ::[num16]"r"(16));//VALU2 IM1(L)
asm volatile("vsll.vx v8, v2, %[num16];" ::[num16]"r"(16));//VALU2 RE0(H)
asm volatile("vsll.vx v30, v4, %[num16];" ::[num16]"r"(16));//VALU2 RE1(H)
asm volatile("vmul.vv v26, v6, v12;::");//VALU1 IM0(L)*IM1(L)
asm volatile("vsrl.vx v10, v8, %[num16];" ::[num16]"r"(16));//VALU2 RE0(L)
asm volatile("vsrl.vx v14, v30, %[num16];" ::[num16]"r"(16));//VALU2 RE1(L)
asm volatile("vmul.vv v16, v10, v14;::");//VALU1 RE0(L)*RE1(L)
asm volatile("vadd.vv v16, v16, v26;::");//VALU1 RE0(L)*RE1(L)+IM0(L)*IM1(L)
asm volatile("vmul.vv v28, v6, v14;::");//VALU1 IM0(L)*RE1(L)
asm volatile("vsetvli %[v1], %[av1],209;:[v1] \"=&r\" (v1):[av1] \"r\" (32));
asm volatile("vredsum.vs v22, v16, v26;::");//VALU2
asm volatile("vsetvli %[v1], %[av1],209;:[v1] \"=&r\" (v1):[av1] \"r\" (64));
asm volatile("vmul.vv v18, v10, v12;::");//VALU1 IM1(L)*RE0(L)
asm volatile("vsub.vv v18, v28, v18;::");//VALU1 IM0(L)*RE1(L)-IM1(L)*RE0(L)
asm volatile("vsetvli %[v1], %[av1],209;:[v1] \"=&r\" (v1):[av1] \"r\" (32));
asm volatile("vredsum.vs v20, v18, v26;::");//VALU2
asm volatile("vsetvli %[v1], %[av1],208;:[v1] \"=&r\" (v1):[av1] \"r\" (1));
asm volatile("vsll.vx v0, v20, %[num16];" ::[num16]"r"(16));//VALU2
asm volatile("vxor.vv v2, v0, v22;::");//VALU1
asm volatile("vmv.x.s %[result], v2;:[result]\"r\"(*pz));//VALU2
asm volatile("vsync %[syncRd],[rs2];:[syncRd]\"=&r\"(syncRd):[rs2]\"r\"(0xFF));
```



OSP (offset scalar product)

OSP (Offset Scalar Product) is defined as the differential autocorrelation in the frequency - domain direction of a total of 180 DMRS, SSS, and the LS estimates obtained from the differences, that is:

$$OSP_{com} = \sum_{l=1}^3 \sum_{k=[0:4:239] \atop +nid\%4} \hat{H}_{LS}(k+1, l) \hat{H}_{LS}^*(k, l)$$

$$OSP_{real} = \max(\text{real}(OSP_{com}), 0)$$

For a total of 180 points of DMRS, SSS, and the interpolated LS estimates, perform the misaligned conjugate multiplication in the frequency - domain. Calibration: SC16.15 * SC16.15 = SC33.30. Accumulate the results of 59*3 = 177 multiplications to obtain ospComm. Calibration: SC40.29. Shift 8 bits to get SC32.22. Take the real part to obtain ospReal:

```
void Test_Osp_Calc()
{
    T_L1R_CSM_ComplexS32    ospRst;

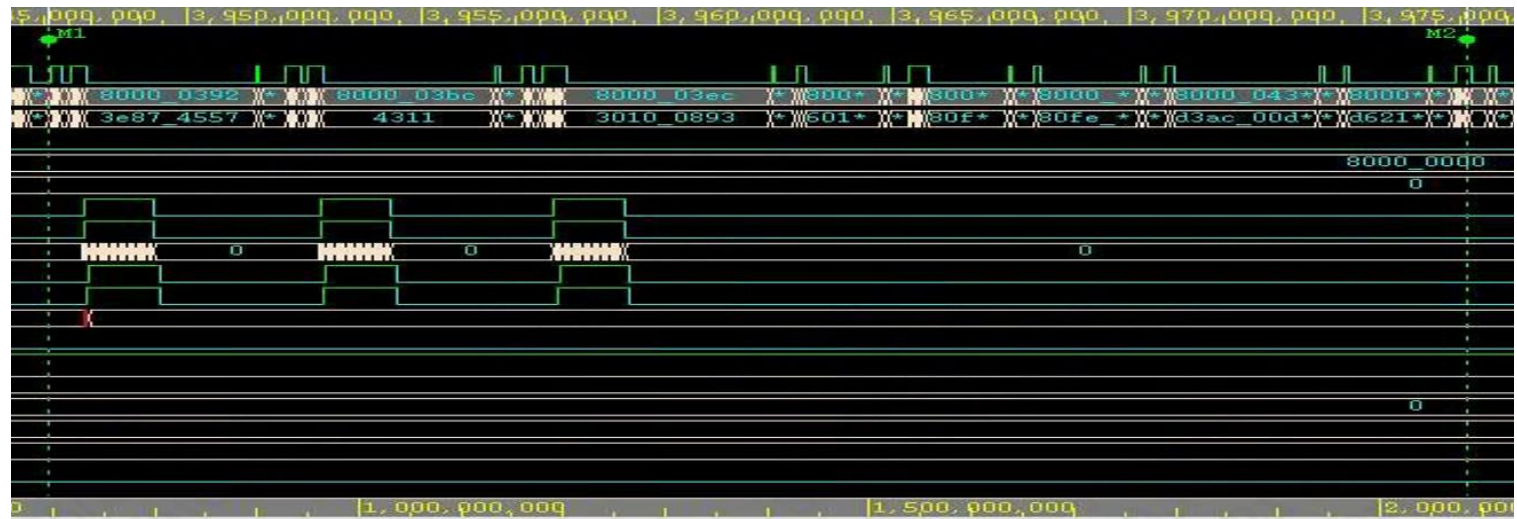
    MCPS_OPEN_FILE
    MCPS_START_CLOCK

    L1R_algo_csm_SsbMeas_CalcOsp((T_L1R_CSM_ComplexS16*)&gOspTstInData

    MCPS_STOP_AND_LOG("Test_Osp_Calc_Cycle ")
    MCPS_CLOSE_FILE
}
```

frcc	00000010	16	frcc	000006af	1711
------	----------	----	------	----------	------

ceva clock = 1711 - 16 = 1695, 16 is start counter, 1711 is end counter, use ceva lib function clock() to get processor clock cycles.



Zvw VPU cycles for OSP = 485

NSP (normal scalar product)

NSP (Normal Scalar Product) is defined as the differential autocorrelation in the time - domain of a total of 180 DMRS, SSS, and the LS estimates obtained from the differences, that is:

$$NSP = \sum_{l=1,2} \sum_{k=[0:4:239] + nid \% 4} \hat{H}_{LS}(k, l) \hat{H}_{LS}^*(k, l + 1) = \sum_{l=1,2} \sum_k H(l, k) H^*(l + 1, k)$$

For a total of 180 points of DMRS, SSS, and the interpolated LS estimates, perform the misaligned conjugate multiplication in the time - domain. Calibration: SC16.15 * SC16.15 = SC33.30. Accumulate 60×2 = 120 products to obtain nsp. Calibrate to SC40.30 and shift 8 bits to get SC32.22:

```
void Test_Nsp_Calc()
{
    T_L1R_CSM_ComplexS32 nspRst;

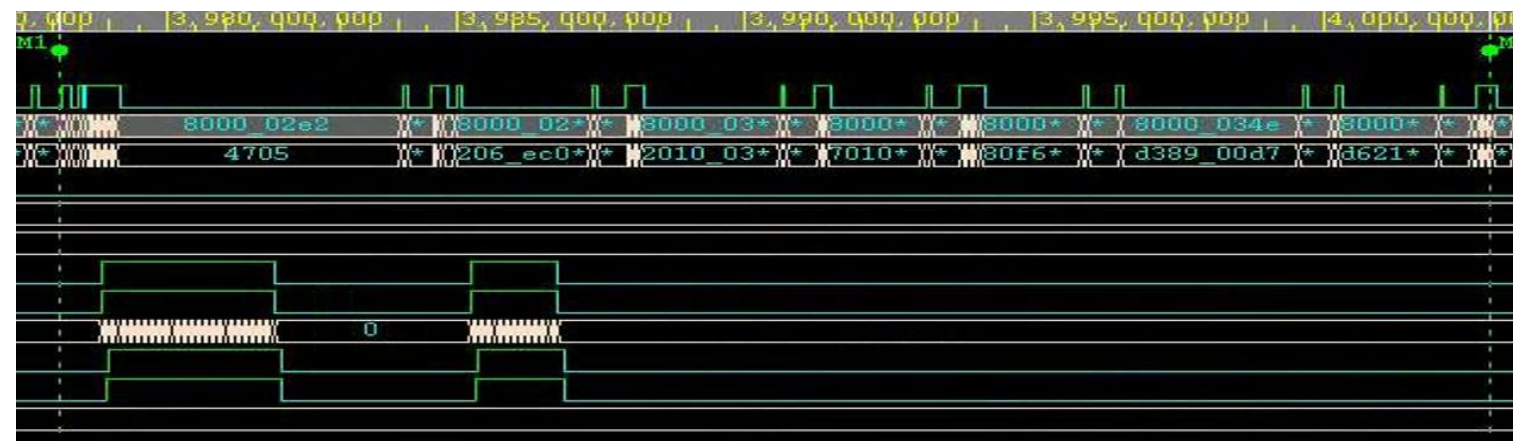
    MCPS_OPEN_FILE
    MCPS_START_CLOCK

    L1R_algo_csm_SsbMeas_CalcNsp((T_L1R_CSM_ComplexS16*) gNspTestInDataRx0[0], &nspRst);

    MCPS_STOP_AND_LOG("Test_Nsp_Calc_Cycle ")
    MCPS_CLOSE_FILE
    return;
}
```

frcc	00000013	19	frcc	0000049b	1179
------	----------	----	------	----------	------

ceva clock = 1179 - 19 = 1160, 19 is start counter, 1179 is end counter, use ceva lib function clock() to get processor clock cycles.



Zvw VPU cycles for NSP = 399